```
  }
 ...
}
```

❶ State that contains the counted words

❷ State that contains the lag and total aggregated words

❸ Timeout for asking

❹ Degree of parallelism read from the configuration

❺ Asks for aggregation to the workers via the router

If the message the master sends to itself is `CountWords`, it adds the count to the total number of `countedWords` using a `merge` function. If the message is `FailedJob`, it sends the failed text to itself and puts it back into the `lag` queue.

Listing 8.11 Master's `working` method implementation, part 2

```
object Master {

  ...
  def working(
      workersRouter: ActorRef[Worker.Command],
      countedWords: Map[String, Int] = Map(),                    ❶
      lag: Vector[String] = Vector()): Behavior[Event] =         ❷
    Behaviors.setup[Event] { context =>

      ...
      Behaviors.receiveMessage[Event] {
        case Tick =>
          ...
        case CountedWords(map) =>                                ❸
          val merged = merge(countedWords, map)
          context.log.debug(s"current count ${merged.toString} ")
          working(workersRouter, merged, lag)
        case FailedJob(text) =>                                  ❹
          context.log.debug(
            s"failed, adding text to lag ${lag.size} ")
          working(workersRouter, countedWords, lag :+ text)
      }
    }

  private def merge(                                             ❺
```